WHAT ABORTING COSTS: A FEASIBILITY INTERVAL AND WASTED-COMPUTE BOUND FOR MULTI-AGENT LLM ORCHESTRATION

Chaitanya Belwal [1] **Girik Malik** [1]

ABSTRACT

Multi-agent LLM systems route work across agents of different priority: an interactive, user-facing agent, a set of periodic background agents (monitors, summarizers, retrievers), and occasional interrupt-driven controllers. We study orchestrators that discard a lower-priority attempt when a higher-priority job preempts it, rather than checkpointing and resuming it. Context assembly, decoding/tool execution, and commit processing are all preemptible; an aborted attempt restarts from scratch. Real-time scheduling theory has studied this behavior in Priority-based Functional Reactive Programming (P-FRP) for safety-critical embedded tasks, where a bounded feasibility interval was derived (Belwal et al., 2013). We reuse that timing model, but separately account for the metered resources consumed by discarded attempts. This paper (1) restates the LCM-based feasibility interval for periodic agent orchestration with an n-agent argument, and (2) derives a closed-form retry-count bound conditional on a valid abort-aware response-time bound, together with a worst-case wasted-compute bound that includes partial commit work. A corrected three-agent trace illustrates a deadline-meeting job that is aborted during commit and wastes 47.1% of its consumed processing time. A counterexample explains why classical resume-based response-time analysis cannot certify abort-and-restart execution. We also examine a sporadic-arrival counterexample and discuss transactionally staged side effects, timestamp ordering, and applicability to real serving systems. Sensitivity analysis and empirical validation are deferred to a larger experimental study.

1 INTRODUCTION

A user asks an assistant a question. While the assistant is drafting a reply, a background agent is in the middle of summarizing yesterday's logs, and a retrieval agent is halfway through re-indexing a document. The user's request is higher priority. The orchestrator stops the summarizer, runs the user-facing agent, and only afterward lets the summarizer continue – or more commonly, lets it restart, because its half-built summary and half-issued tool calls are not something the system trusts enough to resume blindly.

Restarting rather than resuming is a scheduling policy, not by itself a solution to side-effect safety. An LLM agent turn calls external tools, writes to a vector store, sends a message, or modifies a file; discarding its local state does not undo those operations, and retrying can duplicate them. Our baseline therefore assumes read-only tool calls or transactionally staged effects published only at successful completion (Section 2). Many serving engines also reclaim GPU memory, including the KV cache of a preempted request, when a higher priority request arrives (Kwon et al., 2023). Where progress cannot be reused, the scheduling policy studied here aborts the low-priority attempt and retries the same job later, while still charging all resources consumed by the discarded attempt.

Abort-on-preempt scheduling is not new. Priority-based Functional Reactive Programming (P-FRP) (Belwal et al., 2013) uses it for safety-critical embedded control, where a preempted task's intermediate state is discarded rather than preserved because preserving it correctly is either unsafe or too costly, and asks the same question we open with here: does the classical LCM-based feasibility interval still bound the worst case once a preempted job is aborted rather than resumed? We reuse that question as a starting point, not an ending point. The embedded-control setting P-FRP targets has one property agent orchestration does not: an aborted task's wasted work is free. A discarded control-loop iteration burns a few microseconds of CPU time that the hardware was going to spend regardless. A discarded LLM agent turn burns real, metered compute – prompt tokens re-read, output tokens re-generated, tool calls re-issued – every one of which has a dollar cost or an energy cost attached, independent of whether the attempt

[1]Microsoft, USA. Correspondence to: Chaitanya Belwal <cbelwal@microsoft.com>, Girik Malik <girikmalik@microsoft.com>.

Proceedings of the 8th MLSys Conference, Santa Clara, CA, USA, 2025. Copyright 2025 by the author(s).

ever commits. P-FRP's own abort-cost assumption (negligible discard cost, Section 2) is sound for its setting and unsound for ours if we stop at the feasibility interval alone. Sections 3–4 therefore only restate P-FRP's timing result with agent-specific names; the content past that point – the bounded-retry and wasted-compute results of Section 5 and the sporadic/blocking-cost extensions of Sections 6–8 – has no counterpart in (Belwal et al., 2013) and is this paper's actual contribution.

We are not claiming LLM agents and P-FRP tasks are identical. An LLM agent's execution time is not fixed and not tightly bounded the way a hand-written embedded task's WCET is. We address that directly in Section 8. What carries over cleanly from P-FRP is the abort-on-preempt rule and the LCM feasibility window; what does not carry over, and what we add here, is an accounting of what an abort costs beyond lost time – the bound this paper's title refers to, not the timing restatement alone.

Contributions. (1) Two results with no counterpart in the P-FRP line of work, and this paper's central contribution: a closed-form bound on the number of times a job is aborted before it finally completes, and a corollary translating that bound into worst-case wasted compute – and, where attempts are billed, worst-case wasted cost in tokens, tool calls, and dollars burned on attempts that never commit (Section 5). (2) The scaffolding that bound is built on: an agent timing model that reuses the P-FRP job/period/priority structure, renamed for agent orchestration (Section 2), together with a necessary scheduling condition and an LCM feasibility-interval theorem for abort-based preemption, carried over from (Belwal et al., 2013) with a fuller n-agent induction (Sections 3–4). (3) A worked numeric example instantiating both the timing bound and the wasted-compute bound, extended with a sporadic-arrival variant. (4) A discussion of where the fixed-period assumption breaks for real agent workloads, including a direct comparison between abort-based waste and the blocking cost of deferred-preemption scheduling.

2 Agent Timing Model

We reuse the P-FRP notation from (Belwal et al., 2013) with agent-specific names.

Definition 1 (Agent set)**.** *Let $\Gamma_n = \{A_1, A_2, \ldots, A_n\}$ be a set of n recurring agents managed by one orchestrator. Each A_k is triggered periodically: by a polling loop, a cron-style schedule, or (as an approximation, see Section 8) a fixed nominal interval between user turns.*

Definition 2 (Job)**.** *A single run of A_k – one turn, one invocation – is a job of A_k. $A_k(t)$ denotes the job released at time t.*

Definition 3 (Priority)**.** *Each agent A_k has priority k, a positive integer, higher value meaning higher priority. A user-interrupt controller gets the highest priority number in the set; a nightly housekeeping agent gets the lowest.*

Definition 4 (Period, offset)**.** *T_k is the time between successive jobs of A_k. Φ_k is the release time of the first job, so the m-th job of A_k is released at $R_{k,m} = \Phi_k + (m-1)T_k$.*

Definition 5 (Processing time)**.** *C_k is the agent's worst-case processing time for model decoding plus tool execution, taken over the worst-case input the agent is expected to see. $t_{\mathrm{ctx}}(k)$ is the time for assembling the agent's context (prompt construction, retrieval, KV-cache priming). $t_{\mathrm{commit}}(k)$ is the time for preparing and committing staged results to shared state. All three phases occupy the single modeled resource and are preemptible. Externally visible effects are published atomically only at successful completion; atomic visibility does not make commit processing non-preemptible. Total processing time is*

$$P_k = t_{\mathrm{ctx}}(k) + C_k + t_{\mathrm{commit}}(k).$$

Definition 6 (Abort rule)**.** *If a running attempt of A_k is preempted by a strictly higher-priority job before successful completion, the attempt is aborted, whether it is in context assembly, decoding/tool execution, or commit processing. Its partial state and staged effects are discarded. No partial credit carries to the next attempt; the same job restarts from $t_{\mathrm{ctx}}(k)$ on its next opportunity. All processing consumed before the abort, including partial commit work, is waste. Discard/cancellation overhead is assumed negligible; a system with non-negligible cancellation or rollback overhead must model and charge it separately.*

Definition 7 (Deadline)**.** *D_k is how long a job of A_k may take, measured from release, before it is considered late. We take $D_k = T_k$: completion at or before the next release meets the deadline. If a job remains unfinished at its deadline, the baseline schedulability run records a deadline miss and stops. It does not replace, merge, or silently drop the previous job when another job of the same agent arrives. Continuing after a miss requires a separately specified queueing or dropping policy and is outside the baseline model.*

The orchestrator executes at most one attempt across all agents on a single resource, always selecting the highest-priority ready job. Each unfinished job is either ready or running; an aborted job remains ready for a fresh attempt. A strictly higher-priority release immediately aborts the running lower-priority attempt.

Tool-call and side-effect contract. Tool calls are read-only or their externally visible effects are transactionally staged until successful commit. Idempotency alone is not assumed to undo an aborted attempt's effects. Aborted calls remain charged, even when their results are discarded.

Irreversible, unstaged effects and remote calls that continue consuming resources after cancellation are excluded from the baseline; supporting them requires explicit recovery semantics and additional cost accounting.

Timestamp ordering. At each timestamp the baseline simulator must first finalize any successful completion and publish its staged effects, then check deadlines, then batch all releases, and finally dispatch the highest-priority ready job. Simultaneous releases are resolved strictly by priority; no intermediate lower-priority attempts are started within a release batch, and a batch aborts an existing attempt at most once. An attempt completing at time f occupies a half-open interval $[s, f)$, so a release at exactly f does not abort it. A release at a phase boundary before overall completion can still abort the attempt.

3 A Necessary Scheduling Condition

Lemma 1. *Consider $\Gamma_2 = \{A_1, A_2\}$ with A_2 the higher-priority agent. If Γ_2 is schedulable for every release-offset choice, then*

$$T_1 \geq P_1 + P_2 \quad \textit{and} \quad T_2 \geq P_1 + P_2.$$

Proof. Two cases on the relation between T_1 and T_2.

Case $T_2 < T_1$. Release both agents at $t = 0$. A_2 has priority, so it runs first and finishes at P_2. A_1 starts then. The next job of A_2 is due at T_2, so A_1 has only $T_2 - P_2$ time units before it is aborted. If Γ_2 is schedulable, A_1 must finish inside this window every cycle (it gets the same window after every abort, since A_2's jobs are themselves never delayed – they have the top priority). So $P_1 \leq T_2 - P_2$, i.e., $P_1 + P_2 \leq T_2$. Since $T_2 \leq T_1$ by the case assumption, $P_1 + P_2 \leq T_2 \leq T_1$ follows directly.

Case $T_2 \geq T_1$. Release both at $t = 0$. A_2 again finishes first at P_2, A_1 starts and has until T_1 (its own next release) to finish, since the question is whether A_1 meets its own deadline, not whether it beats A_2's next job. So $P_1 \leq T_1 - P_2$, i.e. $P_1 + P_2 \leq T_1$. Because $T_2 \geq T_1$ this also gives $P_1 + P_2 \leq T_2$.

Both cases give $P_1 + P_2 \leq \min(T_1, T_2) \leq T_1$ and $\leq T_2$, hence $T_1, T_2 \geq P_1 + P_2$. □

This is the same shape as the classical preemptive-scheduling capacity bound (a lower-priority job needs room between higher-priority jobs), but it is a necessary, not sufficient, condition here, because the abort rule means a job that *almost* finishes before being preempted gains nothing – unlike classical preemptive scheduling, where partial progress is never lost. A task set can satisfy Lemma 1 and still be unschedulable if the release pattern is adversarial. That is the actual content of the feasibility-interval question: how far out do we have to search over release patterns to be sure.

4 Periodicity of Interference

Fix $\Gamma_2 = \{A_1, A_2\}$, A_2 higher priority, and let $L = \mathrm{lcm}(T_1, T_2)$.

Lemma 2. *If a job of A_1 first starts processing at time t_{start}, another job of A_1 also first starts processing at $t_{\mathrm{start}} + L$.*

Proof. Let a job of A_1 be released at t and another at $t + L$ (this is forced, since L is a multiple of T_1). Suppose a job of A_2 released at t_x is the one that delays $A_1(t)$'s start – the one running (or about to run) when $A_1(t)$ is released, or the one that preempts it. Because L is also a multiple of T_2, A_2 has a job released at $t_x + L$ as well, and no other job of A_2 sits in between in a way that differs from the t_x case, by periodicity of A_2's release pattern over L. Three sub-cases:

$t_x < t$: $A_2(t_x)$ is already running when $A_1(t)$ arrives. It finishes at $t_x + P_2$ and $A_1(t)$ starts then, so $t_{\mathrm{start}} = t_x + P_2$. By the same argument shifted by L, $A_1(t + L)$ starts at $t_x + P_2 + L = t_{\mathrm{start}} + L$.

$t_x = t$: Simultaneous release, A_2 wins on priority, same algebra as above with $t_x = t$.

$t_x > t$: $A_1(t)$ starts immediately on release since nothing is running ahead of it at that instant, so $t_{\mathrm{start}} = t$, and by the shifted argument $A_1(t + L)$ also starts immediately at $t + L$.

All three sub-cases give $t_{\mathrm{start}} + L$ for the second job's start time. □

Lemma 3. *If a job of A_1 completes at time t_{finish} (successfully, without its final attempt being aborted before commit finishes), a job of A_1 also completes at $t_{\mathrm{finish}} + L$.*

Proof. Sketch, by the same shift argument as Lemma 2: every job of A_2 that can interfere with A_1's run between its start and its finish at time t_{finish} has a counterpart exactly L later, because L is a common multiple of both periods. The interference pattern A_1 experiences during $[t_{\mathrm{start}}, t_{\mathrm{finish}})$ is therefore reproduced exactly during $[t_{\mathrm{start}} + L, t_{\mathrm{finish}} + L)$, giving the same elapsed processing and the same finish offset. □

Theorem 1 (Feasibility interval)**.** *For $\Gamma_2 = \{A_1, A_2\}$, the worst-case response time of A_1 over all possible release offsets between A_1 and A_2 occurs at some release pattern within $[0, L)$, $L = \mathrm{lcm}(T_1, T_2)$. Checking schedulability over $[0, L)$ is both necessary and sufficient.*

Proof. By Lemmas 2 and 3, every start/finish pattern A_1 can experience past time L is a repetition, offset by L, of a

pattern that already occurred in $[0, L)$ for some relative offset between Φ_1 and Φ_2 (mod T_1 and T_2 respectively). Since there are only finitely many relative offsets to consider – at most T_1 values of $(\Phi_2 - \Phi_1) \bmod \gcd(T_1, T_2)$-equivalent classes, all realizable inside one L-window once – every achievable response time already appears inside $[0, L)$. Sufficiency: if no deadline is missed anywhere in $[0, L)$ for a given offset, no deadline is missed after L either, since the pattern repeats. Necessity: $[0, L)$ cannot be shortened in general, because the worst relative offset between Φ_1 and Φ_2 can, in principle, only manifest once per L-length window. □

Corollary 1 (n-agent orchestrator)**.** *For* $\Gamma_n = \{A_1, \ldots, A_n\}$*, let* $L = \mathrm{lcm}(T_1, \ldots, T_n)$*. Checking schedulability of every* A_k *over* $[0, L)$*, against interference from every* A_j *with priority* $j > k$*, is sufficient to certify the whole orchestrator.*

Proof. By induction on priority level, from the top down.

Base case, $k = n$ *(top priority).* A_n is never interfered with (no agent has priority $j > n$), so its response time is just P_n and Theorem 1 is vacuous for it; it is trivially schedulable over $[0, L)$ iff it is schedulable at all, since nothing about its timing depends on release offsets.

Inductive step. Suppose the claim holds for every priority above k: for each $j > k$, A_j's worst-case response time is already witnessed inside $[0, L)$. Consider A_k. Let $I_k(t)$ be the total interference A_k receives from all agents of priority $> k$ in the window $[0, t)$ after A_k's release, under the abort rule (every higher-priority job that lands while A_k is running restarts A_k's clock but does not alter its own timing, by the inductive hypothesis). Define the aggregate interfering process as a single virtual periodic source $\hat{A}_k$ with period $\hat{T}_k$ equal to the period of the fastest-recurring combined pattern of $\{A_j : j > k\}$; since each T_j divides L, the combined release pattern of all higher-priority agents is itself periodic with period dividing L (the pattern is a finite union of arithmetic progressions, each with period dividing L, and a finite union of periodic sequences with periods dividing L is periodic with period dividing L). Lemmas 2 and 3, applied with $\hat{A}_k$ in place of A_2 and A_k in place of A_1, give: any start or finish time A_k reaches after t_{start} (or t_{finish}) recurs at $t_{\text{start}} + L$ (or $t_{\text{finish}} + L$). Hence A_k's response time distribution over all release offsets is fully determined by the pattern inside $[0, L)$, which is Theorem 1's statement for the pair $(A_k, \hat{A}_k)$. By induction this holds for every k from n down to 1. □

This still leans on treating the combined higher-priority interference as a single aggregate periodic source rather than deriving its worst-case phase alignment constructively; a fully constructive n-agent proof would need to characterize the worst relative offset among $n - 1$ interfering periods directly, which the original P-FRP paper also left as an extension beyond the two-task case (Belwal et al., 2013).

Experimental interpretation. A hyperperiod is a release-pattern window, not permission to truncate unfinished jobs at its right boundary or to assume that synchronous releases represent every relative phase. Experiments must follow each analyzed job to completion or its deadline, specify initial/carry-in state, and distinguish fixed-offset results from claims over all offsets. Exhaustive offset enumeration is finite only after a time-grid assumption is declared; a sampled continuous-offset sweep is not a proof of global schedulability.

5 Bounded Retries and Wasted Compute

Everything up to this point only reuses (Belwal et al., 2013)'s question (is the system schedulable) renamed for agents. This section asks a second question P-FRP has no reason to ask: when a job is aborted, what did that cost, independent of whether the system is schedulable? For an embedded control task, an aborted iteration costs a few microseconds of CPU time the processor was idle-looping on anyway – not worth bounding separately from the response time itself. For an LLM agent, an aborted attempt has already paid for context assembly and some amount of decoding before it is discarded; those tokens were generated, that tool call may already have fired, and if the serving stack is billed per token or per GPU-second, that cost is sunk and non-refundable the instant the abort happens. The response-time bound of Section 4 says nothing about this, because delay and waste are different quantities that happen to be driven by the same interference pattern.

Definition 8 (Wasted attempt)**.** *A wasted attempt of* A_k *is an execution attempt aborted before successful completion, including an attempt aborted during* $\mathrm{t_{commit}}(k)$*. Its waste is the processing actually consumed before the abort, not the full configured processing time. The final successfully completed attempt is useful work, not waste.*

Lemma 4 (Bounded retry count)**.** *Let* R_k *be a finite, valid upper bound on* A_k*'s response time under the abort-and-restart semantics of Section 2, for the release patterns being analyzed. The number of wasted attempts* N_k *that a single job of* A_k *incurs before it finally completes satisfies*

$$N_k \;\le\; \sum_{j>k} \left\lceil \frac{R_k}{T_j} \right\rceil .$$

Proof. By the abort rule, a running attempt of A_k is wasted exactly when a strictly higher-priority release preempts it before successful completion, including during commit.

Over the half-open interval $[t, t + R_k)$ from release to the completion-time bound, a periodic higher-priority agent A_j releases at most $\lceil R_k/T_j \rceil$ jobs. Each release can trigger at most one abort; simultaneous releases can trigger only one abort of the currently running attempt. Higher-priority carry-in jobs may delay the first start but cannot abort an attempt that has not started. Summing the release counts therefore bounds N_k. A release exactly at completion causes no abort under the timestamp ordering in Section 2. □

The release-count arithmetic also appears in classical fixed-priority response-time analysis (Lehoczky et al., 1989), but its reuse here does not make that analysis abort-aware. A classical recurrence charges higher-priority processing without charging the lower-priority progress destroyed by preemption. Lemma 4 is conditional on an independently established, valid abort-aware R_k; it neither derives that response-time bound nor establishes completion for an unschedulable job. The count is an upper bound, not a claim that every higher-priority release aborts an attempt.

Corollary 2 (Wasted-compute bound)**.** *The processing time A_k consumes on aborted attempts in one response window satisfies*

$$W_k \le N_k P_k \le P_k \sum_{j>k} \left\lceil \frac{R_k}{T_j} \right\rceil,$$

where $P_k = t_{\mathrm{ctx}}(k) + C_k + t_{\mathrm{commit}}(k)$ and R_k satisfies Lemma 4. If π_k is a constant billing rate, or an upper bound on the billing rate, per unit of modeled processing time, billed waste is bounded by $\pi_k W_k$. Token counts and per-call fees require their own per-attempt resource/cost bounds; elapsed time alone does not determine them.

Proof. An aborted attempt can consume context assembly, decoding/tool execution, and part of commit, but no more than P_k processing time. The consumed processing of distinct attempts is disjoint on the single modeled resource, so summing over N_k aborted attempts gives $W_k \le N_k P_k$. Applying Lemma 4 gives the release-count bound. □

Measurement and consistency. For attempt a let $e_{k,a}$ be its actual consumed processing before abort. The simulator records $W_k^{\mathrm{obs}} = \sum_{a \text{ aborted}} e_{k,a}$, including partial commit work, and records useful processing separately. In the deterministic baseline where a successful attempt consumes exactly P_k, the observed response time satisfies $W_k^{\mathrm{obs}} + P_k \le R_k^{\mathrm{obs}}$; waiting and higher-priority execution occupy the remaining time. An upper bound need not be attainable, so $W_k/(W_k + P_k)$ evaluated at a bound is not a measured waste fraction.

The shorter bound $N_k(t_{\mathrm{ctx}}(k) + C_k)$ applies only to a different model in which no aborted attempt consumes commit processing, for example a non-preemptible commit region. Such a region blocks higher-priority jobs and requires revised response-time analysis (Bril et al., 2007); atomic publication alone does not justify excluding commit cost.

A practical use of Corollary 2 is to budget aborted work in addition to useful processing, using independently justified response-time and resource-cost bounds. Section 6 distinguishes observed waste from the conservative bound on a concrete trace.

Table 1. Fixed-offset trace: the first attempt of A_1 is aborted during commit. Times are in seconds.

Interval	Agent	Phase/outcome
$[0, 1.3)$	A_3	Completes
$[1.3, 4.1)$	A_2	Completes
$[6, 6.5)$	A_1	Context assembly
$[6.5, 9.5)$	A_1	Decoding/tools
$[9.5, 10)$	A_1	Partial commit; abort at 10
$[10, 11.3)$	A_3	Completes
$[11.3, 15.8)$	A_1	Fresh attempt; completes

6 Worked Example

Three agents, priority order $A_3 > A_2 > A_1$:

- A_3: interrupt-style user agent. $T_3 = 10$s, $\Phi_3 = 0$, $C_3 = 1$s, $t_{\mathrm{ctx}}(3) = 0.2$s, $t_{\mathrm{commit}}(3) = 0.1$s. $P_3 = 1.3$s.
- A_2: retrieval/index agent. $T_2 = 20$s, $\Phi_2 = 0$, $C_2 = 2$s, $t_{\mathrm{ctx}}(2) = 0.3$s, $t_{\mathrm{commit}}(2) = 0.5$s. $P_2 = 2.8$s.
- A_1: background summarizer. $T_1 = 40$s, $\Phi_1 = 6$s, $C_1 = 3$s, $t_{\mathrm{ctx}}(1) = 0.5$s, $t_{\mathrm{commit}}(1) = 1$s. $P_1 = 4.5$s.

This is a deterministic, hand-worked trace for the stated offsets, not an empirical experiment or a worst-case search over all offsets. The hyperperiod is $L = \mathrm{lcm}(10, 20, 40) = 40$s. At $t = 0$, simultaneous releases are processed by priority: A_3 executes on $[0, 1.3)$ and A_2 on $[1.3, 4.1)$.

A_1 releases at $t = 6$ and begins immediately. It consumes 0.5s of context assembly and 3s of decoding/tool execution, entering commit at $t = 9.5$. The A_3 release at $t = 10$ aborts it after 0.5s of commit processing; no staged effects have been published. After A_3 completes at $t = 11.3$, A_1 restarts and completes at $t = 15.8$. Table 1 lists these events.

The remaining A_3 jobs execute on $[20, 21.3)$ and $[30, 31.3)$, and the second A_2 job executes on $[21.3, 24.1)$. All jobs complete before their deadlines and the resource is idle before the next hyperperiod. The same state and relative release pattern recur every 40s, so this particular offset configuration is schedulable. Its A_1 response time is exactly $R_1 = 15.8 - 6 = 9.8$s; this is not asserted to be a bound for other offsets.

Observed waste versus its bound. The trace has $N_1^{\text{obs}} = 1$ and $W_1^{\text{obs}} = 0.5 + 3 + 0.5 = 4.0$s. Its useful processing is $P_1 = 4.5$s, so the observed waste fraction is $4.0/(4.0 + 4.5) \approx 47.1\%$. The response-time identity is $9.8 = 4.0 + 4.5 + 1.3$s: wasted processing, useful processing, and higher-priority execution. This trace directly demonstrates why partial commit work must be counted; the pre-commit-only expression $N_1^{\text{obs}}(t_{\text{ctx}}(1) + C_1) = 3.5$s would underbound the observed 4.0s.

Using the independently established $R_1 = 9.8$s for this offset configuration, Lemma 4 gives $N_1 \leq \lceil 9.8/10 \rceil + \lceil 9.8/20 \rceil = 2$, and Corollary 2 gives $W_1 \leq 2(4.5) = 9.0$s. These are conservative bounds, not observed values or jointly attainable worst cases. With the observed one abort, the per-attempt bound is $W_1^{\text{obs}} \leq 4.5$s. Under a constant processing-time billing rate π_1, the trace's actual wasted charge is $4.0\pi_1$.

6.1 Why the classical recurrence is insufficient

For contrast, retain the same processing costs but choose $T_3 = 5$s, $T_2 = 8$s, $T_1 = 20$s and synchronous releases. The classical resume-based recurrence (Lehoczky et al., 1989)

$$R_1^{(i)} = P_1 + \sum_{j>1} \left\lceil \frac{R_1^{(i-1)}}{T_j} \right\rceil P_j$$

has the fixed point 14.0s, but that value is not a valid response-time bound under abort-and-restart execution. A_3 occupies 1.3s every 5s, leaving uninterrupted gaps of only 3.7s. Since $P_1 = 4.5$s, A_1 cannot finish any attempt, even if A_2 is removed. Its job misses the deadline at $t = 20$, and the baseline run stops. The pair (A_1, A_3) also violates Lemma 1, since $T_3 = 5 < P_1 + P_3 = 5.8$s. A finite fixed point of the classical recurrence does not account for discarded progress and cannot justify applying the bounded-retry lemma.

6.2 A sporadic variant: the user agent is not actually periodic

A_3 in the corrected trace was given a period $T_3 = 10$s, but interactive turns are generally sporadic. Suppose instead that the UI enforces only a minimum inter-arrival time $T_3^{\min} = 3$s. A periodic stream with releases every 3s is one admissible sporadic sequence. In that sequence A_3 leaves gaps of $3 - 1.3 = 1.7$s, shorter than both $P_2 = 2.8$s and $P_1 = 4.5$s. With simultaneous initial releases of A_2 and A_3, A_2 cannot complete and misses its deadline at 20s. Thus the system cannot guarantee deadlines for all arrivals allowed by this minimum-gap contract.

This periodic counterexample is sufficient to reject a guarantee, but checking only a minimum-gap periodic sequence is not sufficient to certify arbitrary sporadic arrivals in the abort model: phase changes can move interruptions closer to completion. Arbitrary sporadic releases have no repeating LCM window. The retry-count bound still applies with $\lceil R_k/T_j^{\min} \rceil$ when an independent response-time bound valid for all admissible arrivals is available. Design remedies include increasing the enforced minimum inter-arrival time, reducing per-attempt processing, or introducing checkpointing or deferred preemption with a different timing analysis; A_2 is already lower priority than A_3.

7 Experimental Analysis

7.1 Sensitivity Analysis

Sensitivity analysis will be conducted as part of the larger experimental study; no results are reported here. It will examine how schedulability, observed abort counts, consumed processing on aborted attempts (including partial commit), and the observed waste fraction change with the number of higher-priority agents, their periods or minimum inter-arrival times, relative release offsets, and the split of processing time among context assembly, decoding/tool execution, and commit. The experiments will distinguish measured trace outcomes from theoretical upper bounds, assess bound tightness only where a valid abort-aware response-time bound is available, and report deadline misses separately rather than treating unfinished jobs as successful completions.

8 Discussion

Where the fixed-period assumption holds. Polling agents, scheduled retrieval jobs, periodic health checks, and cron-triggered maintenance agents are genuinely periodic. For these, T_k is a real system parameter and the model applies directly.

Where it does not. User-facing turns are not periodic; they arrive when a person types. A minimum inter-arrival contract bounds release counts and supports the sporadic version of Lemma 4, but it does not establish periodicity or make the minimum-gap periodic trace worst-case under abort-and-restart scheduling. A response-time guarantee for arbitrary sporadic arrivals needs separate abort-aware analysis. C_k also depends on prompt length, output length, and tool latency. Hard output-token caps and tool timeouts are prerequisites for finite processing bounds; average latency is not a WCET guarantee. The single-resource baseline does not model asynchronous tool execution overlapping other agents.

Abort cost vs. blocking cost. The classical alternative to aborting a lower-priority job is to defer preemption: let it finish its current non-preemptive region, then switch, paying a bounded blocking term instead of throwing work away

(Bril et al., 2007). That is a different trade entirely. Deferred preemption keeps the lower-priority job's progress and costs the *higher*-priority job some extra delay (the blocking term); the abort rule in this paper keeps the higher-priority job's response time clean and costs the *lower*-priority job all of its progress, quantified directly by the W_k bound of Corollary 2. An orchestrator that can tolerate a bounded delay on high-priority agents should use deferred preemption and Bril et al.'s analysis, not this paper's bound; this paper is for the case where that delay is not tolerable and losing the low-priority job's partial work – the compute Corollary 2 bounds – is the cheaper price to pay.

Where the abort rule itself can fail. Some serving systems support reuse of a prompt prefix or KV-cache segment (Kwon et al., 2023). Full progress-preserving resume calls for classical preemptive analysis; partial reuse instead requires a model of the specific processing retained and recomputed. Corollary 2 describes the fully non-resumable baseline, including commit processing, not a calibrated estimate for partially resumable execution. Neither restarting nor resuming automatically makes irreversible tool side effects safe; the staged-effect contract in Section 2 remains essential.

Practical use of the bounds. For periodic workloads, the feasibility-window argument motivates checking release patterns over a hyperperiod while respecting offset choices, carry-in state, and boundary jobs. It is not a replacement for abort-aware execution accounting. Given an independently justified response-time bound, Corollary 2 supplies a budget for aborted processing, including partial commit. The trace in Section 6 demonstrates the distinction between observed waste and the upper bound without asserting that the bound is tight or globally attainable.

8.1 Limitations

We consolidate here, in one place, the caveats on this paper's scope that are otherwise scattered across the preceding discussion.

- **No empirical validation.** Section 6 is a hand-worked deterministic trace and counterexamples, not a running-system measurement or production trace. Experimental results are not yet available. Sensitivity analysis is reserved for the larger experimental study outlined in Section 7.1, alongside empirical validation with simulator and real-system traces.
- **Fixed periods and bounded processing.** The periodic feasibility argument does not certify arbitrary sporadic arrivals by substituting minimum inter-arrival times into an LCM. The sporadic counterexample in Section 6 disproves a guarantee under one admissible stream, but acceptance requires a separate response-time analysis. All phases need finite processing bounds.
- **Conditional response-time input.** Lemma 4 assumes completion within a valid R_k; it is not a schedulability test. The classical resume-based recurrence is insufficient. The n-agent periodicity argument also requires care about initial state and phase coverage, as noted in Section 4.
- **Preemptible, non-resumable attempts.** An aborted attempt contributes no processing progress to the next one, and its waste includes partial commit. Effects are staged until successful completion; irreversible unstaged effects, non-negligible rollback, and uncancelled remote work require additional semantics and accounting. Partial-resume mechanisms can reduce waste but need a retained-progress model.
- **Single resource, static priorities.** The model is single-accelerator and fixed-priority, following the P-FRP original. It does not cover multi-GPU placement, priority inheritance/inversion, or dynamic (e.g., EDF-style) priority assignment, all of which are realistic features of production agent orchestrators and would require separate analysis.
- **Billing model.** Multiplication by π_k assumes a constant or upper-bounded charge per unit of modeled processing. Token-based prices, fixed tool-call fees, batching, and multiple resource types require separate cost bounds; processing seconds alone do not establish a production bill.

9 Related Work

Liu and Layland's rate-monotonic analysis (Liu & Layland, 1973) is the origin of the LCM-based feasibility interval for classical, non-abort preemptive scheduling; Belwal et al. (Belwal et al., 2013) extended the question to abort-based preemption, which Sections 3–4 restate with agent-specific names. The bounded-retry and wasted-compute results of Section 5 have no counterpart in either paper: both ask whether a deadline is met, neither asks what a discarded attempt cost.

Formal timing analysis of agentic/LLM systems. Until recently, LLM serving schedulers were evaluated empirically, not against a schedulability condition. That has started to change, and two papers do the same kind of analysis for agentic/LLM workloads that this paper does, so they deserve a direct comparison rather than a one-line citation.

- **Chronos** (Marref et al., 2026) is the closest prior work to this paper. It applies classical RT response-time analysis to LLM inference directly: each request is a sporadic prefill job with a TTFT deadline plus a decode task bounded by a TBT SLO, and the paper derives a closed-form worst-case response time (WCRT) bound for TTFT, a TBT capacity condition, and a sound admission-control algorithm, validated on a 44-million-request production trace. The difference from

this paper is the preemption model: Chronos's admission control decides once, at arrival, whether to admit a request, and an admitted job then runs to completion – it does not model a running job being killed mid-execution by a higher-priority arrival. Our abort rule (Section 2) is the case Chronos does not cover: what the WCRT bound becomes when in-flight work, not just queued work, can be discarded. The two results are complementary – Chronos bounds response time under admission control, this paper bounds it under abort-and-restart preemption – and an orchestrator that does both (reject at the door, abort in extreme cases) would need both bounds.

- **Astraea** (Ni et al., 2025) targets multi-stage agentic workflows rather than single LLM calls, scheduling by predicted job-completion time and classifying each stage of an agent's workflow as I/O- or compute-bound to decide what to run next. It does not state a schedulability condition or a worst-case bound, but its multi-stage job model supports a point made in Section 8: an agent's C_k is rarely one atomic cost, it is closer to a chain of sub-costs with different resource profiles, and a tighter version of this paper's bound would need to account for that structure rather than treating C_k as a single number.

Chronos's own related work points further back to where this style of analysis actually comes from, and three of those sources are worth citing directly rather than through Chronos. **LaLaRAND** (Kang et al., 2021) derives WCET bounds and a schedulability test for DNN inference one layer at a time, on real embedded CPU/GPU hardware – the same method this paper uses (decompose a job, bound each piece, compose a test), one level below the agent turn. **Sarathi-Serve** (Agrawal et al., 2024) splits an LLM request's prefill cost into bounded chunks interleaved with other requests' decode steps, giving independent, systems-level evidence (beyond Orca (Yu et al., 2022)) that C_k should eventually be a chain of bounded sub-costs rather than one number. **Lehoczky et al.** (Lehoczky et al., 1989) supplies the classical fixed-priority response-time analysis contrasted with abort-and-restart execution in Section 6; that recurrence is not used as an abort-aware completion bound.

Agent-framework papers used to justify the model. The remaining references are used to justify specific parts of the model in Section 2, each annotated below with the idea borrowed from it.

- **PagedAttention / vLLM** (Kwon et al., 2023) – memory-paged KV cache with preemption and eviction of lower-priority requests. This is the direct real-world instance of $t_{\mathrm{ctx}}(k)$/abort cost: an evicted request's cache is gone, and recomputation from the prompt is the "abort and retry" step modeled in Section 2.
- **Orca** (Yu et al., 2022) – iteration-level scheduling for LLM serving, where new requests can be injected between decoding steps of running ones. Motivates treating C_k as made up of many small sub-steps rather than one atomic block, a refinement not pursued here but noted as future work.
- **ReAct** (Yao et al., 2022) – interleaves reasoning steps with tool calls inside one agent turn. Supports the claim in Section 1 (the tool-call side effects argument): a ReAct-style turn cannot be safely paused mid-interleaving without risking a duplicated or missing tool call.
- **Toolformer** (Schick et al., 2023) – agent decides when to call a tool during generation. Same point as ReAct: the decision of *whether* to call a tool is itself mid-generation state, reinforcing that partial generations are not safely resumable.
- **AutoGen** (Wu et al., 2023) – a framework for orchestrating multiple conversable agents with turn-taking rules. Used here as the concrete picture of "multi-agent orchestrator" in Section 2: AutoGen's group-chat manager is the kind of component that would need a priority and preemption rule layered on top of it to use this paper's result.
- **Generative Agents** (Park et al., 2023) – agents with a memory stream and periodic reflection/summarization steps. This is the concrete source of the "periodic background agent" example (A_1 in Section 6): reflection in that system runs on a schedule, independent of user interaction, exactly the periodic-agent case our model targets.

10 Conclusion

Abort-and-restart scheduling separates deadline satisfaction from the resources consumed by discarded attempts. Under the explicitly stated single-resource, fixed-priority semantics, context assembly, decoding/tool execution, and commit processing are all preemptible, and partial commit work is waste. Given a valid abort-aware response-time bound, Lemma 4 bounds the number of aborts and Corollary 2 bounds their consumed processing by the full per-attempt cost P_k. The corrected trace demonstrates commit-phase waste, while the counterexample shows why classical resume-based response-time analysis cannot establish completion in this model. Transactionally staged effects and explicit event ordering are essential to implementing these semantics. Empirical validation, sensitivity analysis, arbitrary sporadic-arrival guarantees, and tighter per-stage resource bounds remain future work; Section 7 reserves the experimental analysis for the larger study.

REFERENCES

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. Idea used: chunked-prefill, stall-free scheduling splits a single request's prefill cost into near-equal chunks interleaved with other requests' decode steps. Direct evidence, independent of Orca, that an LLM agent's C_k is better modeled as a sequence of bounded sub-costs than one atomic number – cited alongside Orca in the discussion of future refinements.

Belwal, C., Cheng, A. M. K., and Liu, B. Feasibility interval for the transactional event handlers of p-frp. *Journal of Computer and System Sciences*, 79(4):530–541, 2013. Idea used: the source proofs (necessary scheduling condition, periodicity of interference, LCM feasibility interval) for abort-on-preempt task execution, renamed here from P-FRP tasks to agent jobs.

Bril, R. J., Lukkien, J. J., and Verhaegh, W. F. J. Worst-case response time analysis of real-time tasks under fixed-priority scheduling with deferred preemption revisited. In *Proceedings of the 19th Euromicro Conference on Real-Time Systems (ECRTS)*, pp. 269–279, 2007. Idea used: this is the useful contrast case. Deferred preemption (non-preemptive regions) lets a lower-priority job keep whatever progress it made before a higher-priority job's blocking term is paid – the opposite of this paper's abort rule, where progress is thrown away, not preserved. Citing it to make the contrast explicit in the Discussion: blocking terms and abort costs are the two different prices a scheduler can make a low-priority job pay, and they are not interchangeable in the response-time recurrence.

Kang, W., Lee, K., Lee, J., Shin, I., and Chwa, H. S. LaLaRAND: Flexible layer-by-layer CPU/GPU scheduling for real-time DNN tasks. In *Proceedings of the 42nd IEEE Real-Time Systems Symposium (RTSS)*, 2021. Idea used: this is the actual prior art for applying WCET bounds and schedulability analysis to neural-network inference, one layer/sub-task at a time. Same method as this paper (decompose a job into bounded sub-costs, derive a schedulability test), applied one level below the agent – at the DNN layer rather than the agent turn. Cited in Chronos's related work as the nearest real-time-DNN analogue; cited here for the same reason, one level removed.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. Idea used: concrete real-system instance of abort-on-preempt – when a lower-priority request's KV-cache pages are evicted under memory pressure, the request is recomputed from the prompt rather than resumed, matching this paper's abort rule and its context-assembly cost.

Lehoczky, J. P., Sha, L., and Ding, Y. The rate monotonic scheduling algorithm: Exact characterization and average case behavior. In *Proceedings of the 10th IEEE Real-Time Systems Symposium (RTSS)*, pp. 166–171, 1989. Idea used: classical fixed-priority response-time analysis contrasted with abort-and-restart execution in the worked counterexample. Its resume-based recurrence does not account for discarded progress and is not used as an abort-aware completion bound.

Liu, C. L. and Layland, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973. Idea used: origin of the LCM-based feasibility interval and the synchronous critical-instant argument for classical (non-abort) preemptive scheduling, which this paper's abort-based result is contrasted against.

Marref, A., Tarmissi, K., and Chaibi, H. Formal schedulability analysis for LLM inference: TTFT and TBT deadline guarantees via response-time theory. *Frontiers in Computer Science*, 8:1873627, 2026. Closest prior work to this paper: applies classical RT response-time analysis (closed-form WCRT, capacity conditions, admission control) directly to LLM serving, modeling each request as a sporadic prefill job plus a decode task. Idea used / contrast drawn: Chronos assumes non-preemptive chunked-prefill scheduling and keeps admitted jobs running to completion once scheduled – it does not model abort-on-preempt. This paper's feasibility-interval result targets exactly the case Chronos does not: what happens to the bound when a lower-priority job in progress is killed, not just queued, by a higher-priority arrival.

Ni, H., Zhang, J., Li, G., Wang, Z., Wu, R., Zhang, C., and Tan, H. Astraea: A state-aware scheduling engine for LLM-powered agents. *arXiv preprint arXiv:2512.14142*, 2025. Idea used: schedules multi-stage agentic workflows (not single requests) by job-completion-time, using per-stage state/history to classify stages as I/O- vs compute-bound. Supports this paper's framing of an agent "job" as a multi-step, variable-cost unit rather than a single LLM call, referenced in Section 7's discussion of C_k variability.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. Idea used: periodic, schedule-driven reflection/summarization over an agent's memory

stream, independent of direct user interaction – the concrete source for the periodic background-agent example (A_1) in Section 5 of this paper.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. Idea used: the decision of whether to invoke a tool is itself computed mid-generation, reinforcing that partially generated output is not a safe checkpoint to resume from.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023. Idea used: concrete picture of the "orchestrator" in this paper's model – a group-chat / turn-taking manager coordinating several agents is exactly the component that would need a priority and preemption policy layered on top of it.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. Idea used: reasoning and tool-call steps are interleaved inside a single agent turn, which is why a turn cannot be paused and resumed mid-way without risking a duplicated or skipped tool call – the practical justification for the abort rule.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. Idea used: iteration-level scheduling, where new jobs can be admitted between individual decode steps of a running job. Suggests C_k should eventually be modeled as a sequence of sub-steps rather than one atomic block; flagged here as a direction not pursued in this paper.